Tapwave, Inc. Proprietary

Tapwave®

# Tapwave TwGfx Graphics API Reference

| Current Version | Date | Author |
|---|---|---|
| REV 0.3 | 07/27/2003 | MS |
| | | |

## Revision History

| Version | Date | Description | Author |
|---|---|---|---|
| REV 0.1 | 04/30/2003 | | KH |
| REV 0.2 | 06/20/2003 | | KH |
| REV 0.3 | 7/27/2003 | LIGHT COPY EDIT | MS |

© Copyright 2003 Tapwave, Inc. All Rights Reserved. Tapwave is a registered trademark of Tapwave, Inc. Palm OS, the Palm logo, Graffiti, HotSync, and PalmSource are registered trademarks of Palm, Inc. Palm, Palm Powered, and the Palm Powered logo are trademarks of Palm, Inc. X-Forge is a trademark of Fathammer, Ltd. Java is a registered trademark of Sun Microsystems, Inc. Windows is a registered trademark of Microsoft Corporation, Inc. All other brands are trademarks or registered trademarks of their respective owners.

## Background

To use the TwGfx library your application must include TwGfx.h, which is automatically included by Tapwave.h.

You must open the library with TwGfxOpen before making any other calls, and you must call TwGfxClose before your application exits. You can call TwGfxOpen as many times as desired, but you must pair each call with a matching TwGfxClose.

The library provides facilities for the allocation and manipulation of surfaces, objects which are rectangular regions of memory associated with the graphics accelerator. In addition to basic rendering operations (points, lines, rectangles) there are a large set of *bit blt* operations used to perform surface to surface copy operations. Note that bit blt operations that use the same surface for both the source and destination will yield undefined results in the overlapping area.

Surface memory can be accessed directly by the CPU. However, this access is not as efficient as accessing CPU memory. To increase performance several *asynchronous* procedures are provided to transfer data to/from a surface using background DMA. This allows the CPU to continue executing while the copy operation proceeds. When such a transfer is started (see TwGfxReadSurface or TwGfxWriteSurface later in this document) the surface is considered *busy*. Any other attempt to use the surface will fail, returning a twGfxErrorOperationInProgress error. To determine when a surface is no longer busy use the TwGfxIsSurfaceReady procedure.

In addition to the asynchronous copy operation, there is also an *asynchronous bit blt* operation. This is used to avoid waiting for vertical blank before issuing a bitblt, thus freeing up the CPU to perform other calculations. There can be at most one asynchronous bitblt going at any time, for all instances of the library.

## Library data types

```
typedef struct TwGfxInfoType {
  /* caller MUST set this to sizeof (TwGfxInfoType) */
  Int32 size;

  Int32 displayWidth, displayHeight;    /* current dimensions of display */
  Int32 displayRowBytes;                /* byte width of entire row */
  Int32 displayPixelFormat;             /* format of display */

  Int32 freeAcceleratorMemory;          /* free accelerator memory */
  Int32 totalAcceleratorMemory;         /* total accelerator memory */
} TwGfxInfoType;

/*
 * Surface information structure
 */
typedef struct TwGfxSurfaceInfoType {
  /* caller MUST set this to sizeof(TwGfxSurfaceInfoType) */
  Int32 size;

  Int32 width, height;                  /* dimensions */
  Int32 rowBytes;                       /* byte width of entire row */

  Int32 location;                       /* memory location of the surface */
  Int32 pixelFormat;                    /* format of the surface */
} TwGfxSurfaceInfoType;
```

```
/*
 * Bitmap structure
 */
typedef struct TwGfxBitmapType {
  /* caller MUST set this to sizeof(TwGfxBitmapType) */
  Int32 size;

  Int32 width, height;                /* size of bitmap */
  Int32 rowBytes;                     /* bytes per row */
  Int32 pixelFormat;                  /* format of data */
  void* data;                         /* actual data */

  UInt16* displayPalette;             /* In native display format */
  TwGfxPackedRGBType transparentColor;
} TwGfxBitmapType;

typedef struct TwGfxPointType {
  Int32 x, y;
} TwGfxPointType;

typedef struct TwGfxRectType {
  Int32 x, y, w, h;
} TwGfxRectType;

typedef struct TwGfxSpanType {
  Int32 x, y, w;
} TwGfxSpanType;
```

## Library macros

```
/*
 * Macro to construct a frame buffer compatible rgb565 color given
 * three 8 bit components. This is a manually optimized version that
 * is consistent with the other color macros here.
 */
#define TwGfxMakeDisplayRGB(_r,_g,_b) \
  ( (((_r) & 0xF8) << 8) | (((_g) & 0xFC) << 3) | (((_b) & 0xF8) >> 3) )

/*
 * These macros take an 8 bit color component and adjust the
 * size to match the rgb565 display framebuffer
 */
#define TwGfxRComponentToDisplayComponent(_r) (((_r) & 0xF8) >> 3)
#define TwGfxGComponentToDisplayComponent(_g) (((_g) & 0xFC) >> 2)
#define TwGfxBComponentToDisplayComponent(_b) (((_b) & 0xF8) >> 3)

/*
 * This macro converts from packed component RGB to packed display RGB
 */
#define TwGfxPackedRGBToDisplayRGB(_rgb)                                \
  ( (TwGfxRComponentToDisplayComponent((_rgb) >> 16) << twGfxRShift) | \
    (TwGfxGComponentToDisplayComponent((_rgb) >> 8) << twGfxGShift) |  \
    (TwGfxBComponentToDisplayComponent((_rgb)) << twGfxBShift) )

/*
 * This macro converts a TwGfxRGBType structure to a
 * TwGfxPackedRGBType value.
 */
#define TwGfxRGBToPackedRGB(_rgb) \
  ((TwGfxPackedRGBType) ( ((_rgb).r << 16) | ((_rgb).g << 8) | (_rgb).b ) )
```

```
/*
 * This macro converts rgb components to a TwGfxPackedRGBType
 */
#define TwGfxComponentsToPackedRGB(_r,_g,_b)       \
  ((TwGfxPackedRGBType) ( (((_r) & 0xFF) << 16) | \
                          (((_g) & 0xFF) << 8) |  \
                          ((_b) & 0xFF) ) )
```

# Library access calls

### TwGfxOpen

| | | |
|---|---|---|
| **Purpose** | Access the accelerated graphics library. You can make this call as many times as desired. However, you must pair each TwGfxOpen with a TwGfxClose call to avoid resource leaks. | |
| **Prototype** | `Err TwGfxOpen(TwGfxType** aResult,`<br>`TwGfxInfoType* aInfoResult)` | |
| **Fields** | `[out] aResult` | Pointer to a handle to the graphics library. If the call succeeds then *aResult is set to a handle to the graphics library for use in subsequent calls. |
| | `[inout] aInfoResult` | Pointer to a TwGfxInfoType object which is filled with a description of the capabilities of the device and the graphics library. Note that you must set the size field of TwGfxInfoType to the size of the data structure.<br><br>This argument and the size field can also be NULL if no such data is desired. |
| **Result** | `errNone` – Succeeded<br><br>TwGfxErrorLibraryOpen – the library is already open<br><br>twGfxErrorBadObjectVersion – the TwGfxInfoType size field doesn't match a known version for the library | |
| **Side Effects** | | |
| **Comments** | | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

### TwGfxClose

| | | |
|---|---|---|
| **Purpose** | Shutdown use of the library, releasing all resources associated with the library. | |
| **Prototype** | `Err TwGfxClose(TwGfxType* aGfx)` | |
| **Fields** | `[in] aGfx` | A handle to the graphics library. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to the library was invalid | |
| **Side Effects** | This call releases all resources allocated by the library including any surface objects that were created and not yet released. Note that only the surfaces created using this instance of the TwGfx library are released. Continuing to use the aGfx handle after calling close yields undefined results (most likely a crash). | |
| **Comments** | | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxGetInfo**

| | | |
|---|---|---|
| **Purpose** | Query the graphics library for information describing the capabilities of the device and of the graphics library. | |
| **Prototype** | `Err TwGfxGetInfo(TwGfxType* aGfx, TwGfxInfoType* aInfoResult)` | |
| **Fields** | `[in] aGfx` | A handle to the graphics library. |
| | `[inout] aInfoResult` | Pointer to a TwGfxInfoType object which will be filled in with a description of the capabilities of the device and of the graphics library. The size field of TwGfxInfoType must be set to the size of the data structure. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to the library is invalid<br><br>twGfxErrorNullPointer – the aInfoResult pointer is NULL<br><br>twGfxErrorBadObjectVersion – the TwGfxInfoType size field doesn't match a known version for the library | |
| **Side Effects** | | |
| **Comments** | | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

### TwGfxGetMemoryUsage

| | | |
|---|---|---|
| **Purpose** | Query the current usage of graphics accelerator memory. | |
| **Prototype** | `Err TwGfxGetMemoryUsage(TwGfxType* aGfx, Int32 aLocation, Int32* aUsedResult)` | |
| **Fields** | `[in] aGfx` | A handle to the graphics library. |
| | `[in] aLocation` | Must be twGfxLocationAcceleratorMemory |
| | `[out] aUsedResult` | Pointer to an integer that receives the usage value. |
| **Result** | `errNone` – Succeeded<br><br>`twGfxErrorInvalidHandle` – the handle to the library is invalid<br><br>`twGfxErrorNullPointer` – the aUsedResult pointer is NULL | |
| **Side Effects** | | |
| **Comments** | | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxGetDisplaySurface**

| | | |
|---|---|---|
| **Purpose** | Query the graphics library for the surface used for display. The call returns the TwGfxSurfaceType handle that represents the actual display. | |
| **Prototype** | `Err TwGfxGetDisplaySurface(TwGfxType* aGfx, TwGfxSurfaceType** aResult)` | |
| **Fields** | `[in] aGfx` | A handle to the graphics library. |
| | `[out] aResult` | Pointer to a handle to a graphics surface that is filled in when the call succeeds. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to the library is invalid<br><br>twGfxErrorNullPointer – the aResult pointer is NULL | |
| **Side Effects** | | |
| **Comments** | | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

### TwGfxGetPalmDisplaySurface

| | | |
|---|---|---|
| **Purpose** | Query the graphics library for the surface used for the PalmOS display. The call returns the TwGfxSurfaceType handle that represents the actual display. | |
| **Prototype** | `Err TwGfxGetPalmDisplaySurface(TwGfxType* aGfx, TwGfxSurfaceType** aResult)` | |
| **Fields** | `[in] aGfx` | A handle to the graphics library. |
| | `[out] aResult` | Pointer to a handle to a graphics surface that is filled in when the call succeeds. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to the library is invalid<br><br>twGfxErrorNullPointer – the aResult pointer is NULL | |
| **Side Effects** | | |
| **Comments** | The Palm display surface represents the subset of the display surface used by the PalmOS. It stays consistent with the size and location of the PalmOS display. | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxInVBlank**

| | | |
|---|---|---|
| **Purpose** | Query the graphics library and see whether or not the display is in the vertical blanking period. | |
| **Prototype** | `Err TwGfxInVBlank(TwGfxType* aGfx, Boolean* aInVBlankResult)` | |
| **Fields** | `[in] aGfx` | A handle to the graphics library. |
| | `[out] aInVBlankResult` | Pointer to a Boolean value which indicates the current state of the vertical blanking. The value is *true* if the display is in vertical blanking, *false* otherwise. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to the library is invalid<br><br>twGfxErrorNullPointer – the aInfoResult pointer is NULL | |
| **Side Effects** | | |
| **Comments** | | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxWaitForVBlank**

| | | |
|---|---|---|
| **Purpose** | Wait for the vertical blanking period to begin. This call may return immediately if the display is already in the vertical blanking period. | |
| **Prototype** | `Err TwGfxWaitForVBlank(TwGfxType* aGfx)` | |
| **Fields** | `[in] aGfx` | A handle to the graphics library. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to the library is invalid | |
| **Side Effects** | | |
| **Comments** | | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

# Surface calls

### TwGfxAllocSurface

| | | |
|---|---|---|
| **Purpose** | Attempt to allocate a new surface. Surfaces are used by the rendering calls described below. Surfaces should be allocated in accelerator memory. | |
| **Prototype** | `Err TwGfxAllocSurface(TwGfxType* aGfx, TwGfxSurfaceType** aResult, TwGfxSurfaceInfoType* aDescription);` | |
| **Fields** | `[in] aGfx` | A handle to the graphics library. |
| | `[out] aResult` | Pointer to a handle to a surface that is filled in if the call succeeds. |
| | `[inout] aDescription` | Pointer to a TwGfxSurfaceInfoType object. This pointer must not be NULL and the size field must be initialized to the size of the data structure. In addition, you must set the width, height, pixelFormat and location fields to the values desired for creation of the surface. All other fields are ignored. However, when the call succeeds the remaining fields are filled in so that the caller receives a complete description of the surface. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to the library is invalid<br><br>twGfxErrorOutOfMemory – the library ran out of memory<br><br>twGfxErrorInvalidPixelFormat – the pixelFormat field is invalid<br><br>twGfxErrorInvalidLocation – the location field is invalid<br><br>twGfxErrorInvalidSize – the width/height fields are <= zero.<br><br>twGfxErrorNullPointer – the aDescription pointer is NULL<br><br>twGfxErrorBadObjectVersion – the TwGfxSurfaceInfoType size field doesn't match a known version for the library<br><br>twGfxErrorSurfaceAllocFailed – the allocation failed (insufficient device memory is the most likely reason). | |
| **Side Effects** | | |
| **Comments** | Only the pixel format twGfxPixelFormatRGB565_LE can be used for surface creation at this time. | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

### TwGfxFreeSurface

| | | |
|---|---|---|
| **Purpose** | Free a previously allocated surface. | |
| **Prototype** | `Err TwGfxFreeSurface(TwGfxType* aGfx, TwGfxSurfaceType* aSurface)` | |
| **Fields** | `[in] aGfx` | A handle to the graphics library. |
| | `[in] aSurface` | Handle to the surface that should be freed. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to the library or surface is invalid<br><br>twGfxErrorInvalidSurface – the surface refers to the display surface which cannot be freed. | |
| **Side Effects** | | |
| **Comments** | | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxSetClip**

| | | |
|---|---|---|
| **Purpose** | Set the clipping rectangle associated with the given surface. All rendering calls are clipped by the surface clipping rectangle. | |
| **Prototype** | `Err TwGfxSetClip(TwGfxSurfaceType* aSurface, const TwGfxRectType* aClipRect)` | |
| **Fields** | `[in] aSurface` | A handle to the surface. |
| | `[in] aClipRect` | Pointer to the clipping rectangle. If aClipRect is NULL then clipping is disabled for the surface (this has the same effect as setting the clipping rectangle to cover the entire surface). |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to the surface is invalid | |
| **Side Effects** | | |
| **Comments** | | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

### TwGfxGetClip

| | | |
|---|---|---|
| **Purpose** | Get the clipping rectangle associated with a surface. | |
| **Prototype** | `Err TwGfxGetClip(TwGfxSurfaceType* aSurface, TwGfxRectType* aResult)` | |
| **Fields** | `[in] aSurface` | A handle to the surface. |
| | `[out] aClipRect` | Pointer to the rectangle object which will be filled in with the surface clipping rectangle. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to the surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – the aClipRect pointer is NULL | |
| **Side Effects** | | |
| **Comments** | If the clipping for a surface is disabled (the default state for a surface) then the fields in aClipRect will contain the size of the surface (0, 0, width, height). | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxGetSurfaceInfo**

| | | |
|---|---|---|
| **Purpose** | Query the surface information for a given surface. | |
| **Prototype** | `Err TwGfxGetSurfaceInfo(TwGfxSurfaceType* aSurface, TwGfxSurfaceInfoType* aResult)` | |
| **Fields** | `[in] aSurface` | A handle to the surface. |
| | `[inout] aResult` | A TwGfxSurfaceInfoType object that will be filled in with the description of the surface. The size field of aResult must be set to the size of the data structure before making this call. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to the surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – the aClipRect pointer is NULL<br><br>twGfxErrorBadObjectVersion – the TwGfxSurfaceInfoType size field doesn't match a known version for the library | |
| **Side Effects** | | |
| **Comments** | | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxLockSurface**

| | | |
|---|---|---|
| **Purpose** | Return a memory address for the surfaces video memory. | |
| **Prototype** | `Err TwGfxLockSurface(TwGfxSurfaceType* aSurface, void** aAddressResult)` | |
| **Fields** | `[in] aSurface` | A handle to the surface. |
| | `[out] aAddressResult` | A pointer to a void pointer that will be filled in with a readable/writable memory address for the surface memory. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to the surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – the aAddressResult pointer is NULL | |
| **Side Effects** | | |
| **Comments** | The memory address returned remains valid until TwGfxUnlockSurface is called. Note that you can nest these calls. When this occurs, the same memory address is always returned and is valid until the matching number of TwGfxUnlockSurface calls are made.<br><br>Use care when mixing direct access to the surface memory with surface rendering operations. Surface rendering operations can execute in parallel with the CPU's access to the memory and yield unpredictable results. | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

### TwGfxUnlockSurface

| | | |
|---|---|---|
| **Purpose** | Unlock a previously locked surface. | |
| **Prototype** | `Err TwGfxUnlockSurface(TwGfxSurfaceType* aSurface, Boolean aUpdate)` | |
| **Fields** | `[in] aSurface` | A handle to the surface. |
| | `[in] aUpdate` | This flag indicates to the graphics library that the surface memory was modified by the caller. If for some reason actual surface memory was not returned by TwGfxLockSurface, it should be copied to the actual surface memory. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to the surface is invalid or the library is not open<br><br>twGfxErrorSurfaceNotLocked – the surface was not locked | |
| **Side Effects** | | |
| **Comments** | | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxReadSurface**

| | | |
|---|---|---|
| **Purpose** | Make a copy of the surface display memory. | |
| **Prototype** | `Err TwGfxReadSurface(TwGfxSurfaceType* aSurface, void* aDest, Boolean aAsync)` | |
| **Fields** | `[in] aSurface` | A handle to the surface. |
| | `[out] aDest` | A pointer to the memory where the surface memory will be written. It is the callers responsibility to allocate enough memory. |
| | `[in] aAsync` | When true this flag indicates that the copy should be done asynchronously. Use TwGfxSurfaceIsReady to query when the copy is done. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to the surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – the aDest pointer is NULL<br><br>twGfxErrorOperationInProgress – a TwGfxSurfaceRead or TwGfxSurfaceWrite call is already in progress | |
| **Side Effects** | | |
| **Comments** | This is identical to calling TwGfxReadSurfaceRegion with a bound rectangle that covers the entire surface.<br><br>To allocate enough memory for the copy, use TwGfxGetSurfaceInfo and perform this calculation:<br><br>`Int32 bytesNeeded = info.rowBytes * info.height;` | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

### TwGfxReadSurfaceRegion

| | | |
|---|---|---|
| **Purpose** | Make a copy of the surface display memory. | |
| **Prototype** | `Err TwGfxReadSurfaceRegion(TwGfxSurfaceType* aSurface, const TwGfxRectType* aBounds, void* aDestPixels, Int32 aDestRowBytes, Boolean aAsync)` | |
| **Fields** | `[in] aSurface` | A handle to the surface. |
| | `[in] aBounds` | A pointer to the rectangle object which contains the area of the surface to be written. |
| | `[out] aDestPixels` | A pointer to the memory where the surface memory will be written. It is the callers responsibility to allocate enough memory. |
| | `[in] aDestRowBytes` | The number of data bytes per row in aDestPixels. |
| | `[in] aAsync` | When true this flag indicates that the copy should be asynchronous. Use TwGfxSurfaceIsReady to query when the copy is done. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to the surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – the aDest pointer is NULL<br><br>twGfxErrorOperationInProgress – an asynchronous operation is in progress | |
| **Side Effects** | | |
| **Comments** | | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

### TwGfxWriteSurface

| | | |
|---|---|---|
| **Purpose** | Write a memory buffer to the surface display memory. | |
| **Prototype** | `Err TwGfxWriteSurface(TwGfxSurfaceType* aSurface, const void* aSource, Boolean aAsync)` | |
| **Fields** | `[in] aSurface` | A handle to the surface. |
| | `[in] aSource` | A pointer to the memory to be copied to the surface display memory. |
| | `[in] aAsync` | When true this flag indicates that the copy should be asynchronous. Use TwGfxIsSurfaceReady to query when the copy is done. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to the surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – the aSource pointer is NULL<br><br>twGfxErrorOperationInProgress – an asynchronous operation is already in progress | |
| **Side Effects** | | |
| **Comments** | See `TwGfxReadSurface` for how to allocate enough memory for the copy. This call is the same as calling `TwGfxWriteSurfaceRegion` and specifying a region that covers the entire surface with source row bytes that are the same as the surface row bytes. | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

### TwGfxWriteSurfaceRegion

| | | |
|---|---|---|
| **Purpose** | Write a memory buffer to a subset of the surface display memory. | |
| **Prototype** | `Err TwGfxWriteSurfaceRegion(TwGfxSurfaceType* aSurface, const TwGfxRectType* aBounds, const void* aSourcePixels, Int32 aSourceRowBytes, Boolean aAsync)` | |
| **Fields** | `[in] aSurface` | A handle to the surface. |
| | `[in] aBounds` | A pointer to the rectangle object which contains the area of the surface to be written. |
| | `[in] aSourcePixels` | A pointer to the memory to be copied to the surface display memory. |
| | `[in] aSourceRowBytes` | The number of bytes per row of data in aSourcePixels. |
| | `[in] aAsync` | When true this flag indicates that the copy should be asynchronous. Use TwGfxIsSurfaceReady to query when the copy is done. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to the surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – the aSource pointer is NULL<br><br>twGfxErrorInvalidCoord – the coordinates in aDestRect are outside the bounds of the surface or specify an empty area (zero width/height)<br><br>twGfxErrorOperationInProgress – a TwGfxSurfaceRead or TwGfxSurfaceWrite call is already in progress | |
| **Side Effects** | | |
| **Comments** | The amount of memory necessary for the copy to work properly depends on the pixel format of the surface and the height of the aDestRect and aSourceRowBytes. For example, assuming 2 bytes per pixel, the following calculation will be correct:<br><br>totalBytes = aDestRect->h * aSourceRowBytes * 2; | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxIsSurfaceReady**

| | | |
|---|---|---|
| **Purpose** | Query the surface and see if it is ready for another TwGfxSurfaceRead or TwGfxSurfaceWrite call. | |
| **Prototype** | `Err TwGfxIsSurfaceReady(TwGfxSurfaceType* aSurface)` | |
| **Fields** | `[in] aSurface` | A handle to the surface. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to the surface is invalid or the library is not open<br><br>twGfxErrorOperationInProgress – a TwGfxSurfaceRead or TwGfxSurfaceWrite call is still in progress | |
| **Side Effects** | | |
| **Comments** | This call returns errNone when no operations are pending (and the surface is a valid surface). If the arguments are valid and an operation is pending, then it returns twGfxErrorOperationInProgress. | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

## Rendering calls

This section describes the rendering methods in the graphics library.

**TwGfxBitBlt**

| | | |
|---|---|---|
| **Purpose** | Basic bitblt rendering call. | |
| **Prototype** | `Err TwGfxBitblt(TwGfxSurfaceType* aDestSurface, const TwGfxPointType* aDestPoint, TwGfxSurfaceType* aSourceSurface, const TwGfxRectType* aSourceRect)` | |
| **Fields** | `[in] aDestSurface` | A handle to the surface to which you want to bitblt. |
| | `[in] aDestPoint` | The upper-left coordinate in the destination surface to which you want to bitblt. |
| | `[in] aSourceSurface` | A handle to the surface from which you want to bitblt. |
| | `[in] aSourceRect` | The area in the source surface from which you want to bitblt. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to a surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – aDestPoint or aSourceRect is NULL<br><br>twGfxErrorInvalidCoord – the coordinate values in the destination point or the source rectangle address values outside their respective surfaces<br><br>twGfxErrorOperationInProgress – an asynchronous operation is in progress | |
| **Side Effects** | | |
| **Comments** | | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxAsyncBlt**

| | | |
|---|---|---|
| **Purpose** | Basic bitblt rendering call, done asynchronously. | |
| **Prototype** | `Err TwGfxAsyncblt(TwGfxSurfaceType* aDestSurface, const TwGfxPointType* aDestPoint, TwGfxSurfaceType* aSourceSurface, const TwGfxRectType* aSourceRect)` | |
| **Fields** | `[in] aDestSurface` | A handle to the surface to which you want to bitblt. |
| | `[in] aDestPoint` | The upper-left coordinate in the destination surface to which you want to bitblt. |
| | `[in] aSourceSurface` | A handle to the surface from which you want to bitblt. |
| | `[in] aSourceRect` | The area in the source surface from which you want to bitblt. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to a surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – aDestPoint or aSourceRect is NULL<br><br>twGfxErrorInvalidCoord – the coordinate values in the destination point or the source rectangle address values outside their respective surfaces<br><br>twGfxErrorOperationInProgress – an asynchronous operation is in progress or another asynchronous bitblt has been requested | |
| **Side Effects** | | |
| **Comments** | This call is identical to TwGfxBitBlt except that the bitblt occurs during the next vertical retrace period.<br><br>There can only one asynchronous bitblt request at a time. | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxTransparentBlt**

| | | |
|---|---|---|
| **Purpose** | Transparent bitblt rendering call. | |
| **Prototype** | `Err TwGfxTransparentBlt(TwGfxSurfaceType* aDestSurface, const TwGfxPointType* aDestPoint, TwGfxSurfaceType* aSourceSurface, const TwGfxRectType* aSourceRect, TwGfxPackedRGBType aTransparentColor)` | |
| **Fields** | `[in] aDestSurface` | A handle to the surface to which you want to bitblt. |
| | `[in] aDestPoint` | The upper-left coordinate in the destination surface to which you want to bitblt. |
| | `[in] aSourceSurface` | A handle to the surface from which you want to bitblt. |
| | `[in] aSourceRect` | The area in the source surface from which you want to bitblt. |
| | `[in] aTransparentColor` | The color which should not be rendered. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to a surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – aDestPoint or aSourceRect is NULL<br><br>twGfxErrorInvalidCoord – the coordinate values in the destination point or the source rectangle address values outside their respective surfaces<br><br>twGfxErrorOperationInProgress – an asynchronous operation is in progress | |
| **Side Effects** | | |
| **Comments** | The source surface is bitblt'd to the destination surface like a regular bitblt with one distinction: any pixel in the source surface whose color is identical to aTransparentColor is not written to the destination surface. | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

### TwGfxMaskBlt

| | | |
|---|---|---|
| **Purpose** | Mask bitblt rendering call. | |
| **Prototype** | `Err TwGfxMaskBlt(TwGfxSurfaceType* aDestSurface, const TwGfxPointType* aDestPoint, TwGfxSurfaceType* aSourceSurface, const TwGfxRectType* aSourceRect, const TwGfxBitmapType* aMask)` | |
| **Fields** | `[in] aDestSurface` | A handle to the surface to which you want to bitblt. |
| | `[in] aDestPoint` | The upper-left coordinate in the destination surface to which you want to bitblt. |
| | `[in] aSourceSurface` | A handle to the surface from which you want to bitblt. |
| | `[in] aSourceRect` | The area in the source surface from which you want to bitblt. |
| | `[in] aMask` | A monochrome bitmap that describes which pixels to render and which to not render. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to a surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – aDestPoint or aSourceRect is NULL<br><br>twGfxErrorInvalidCoord – the coordinate values in the destination point or the source rectangle address values outside their respective surfaces<br><br>twGfxErrorOperationInProgress – an asynchronous operation is in progress | |
| **Side Effects** | | |
| **Comments** | Only monochrome bitmaps can be used as masks. When a one bit is present in the bitmap the source surface pixel will be written to the destination surface. When a zero bit is present the destination pixel will not be overwritten. | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxBlendBlt**

| | | |
|---|---|---|
| **Purpose** | Blending bitblt rendering call. | |
| **Prototype** | `Err TwGfxBlendBlt(TwGfxSurfaceType* aDestSurface, const TwGfxPointType* aDestPoint, TwGfxSurfaceType* aSourceSurface, const TwGfxRectType* aSourceRect, TwGfxPackedRGBType aSourceAlpha)` | |
| **Fields** | `[in] aDestSurface` | A handle to the surface to which you want to bitblt. |
| | `[in] aDestPoint` | The upper-left coordinate in the destination surface to which you want bitblt. |
| | `[in] aSourceSurface` | A handle to the surface from which you want to bitblt. |
| | `[in] aSourceRect` | The area in the source surface from which you want to bitblt. |
| | `[in] aSourceAlpha` | The constant alpha color to use over the entire surface |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to a surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – aDestPoint or aSourceRect is NULL<br><br>twGfxErrorInvalidCoord – the coordinate values in the destination point or the source rectangle address values outside their respective surfaces<br><br>twGfxErrorOperationInProgress – an asynchronous operation is in progress | |
| **Side Effects** | | |
| **Comments** | The source surface is blended into the destination surface using the following formula on a per pixel basis:<br><br>dst.r = src.r * alpha.r + (1 – alpha.r) * dst.r<br>dst.g = src.g * alpha.g + (1 – alpha.g) * dst.g<br>dst.b = src.b * alpha.b + (1 – alpha.b) * dst.b<br><br>The alpha color componets are logically normalized to a 0.0 – 1.0 range (inclusive) before the above calculation is done.<br><br>The above formula can be translated to “As the alpha value increases towards 1, more of the source pixel is used and less of the destination pixel is used.” | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxMaskBlendBlt**

| | | |
|---|---|---|
| **Purpose** | Maked blending bitblt rendering call. | |
| **Prototype** | `Err TwGfxMaskBlendBlt(TwGfxSurfaceType* aDestSurface, const TwGfxPointType* aDestPoint, TwGfxSurfaceType* aSourceSurface, const TwGfxRectType* aSourceRect, const TwGfxBitmapType* aAlphaMask)` | |
| **Fields** | `[in] aDestSurface` | A handle to the surface to which you want bitblt. |
| | `[in] aDestPoint` | The upper-left coordinate in the destination surface to which you want bitblt. |
| | `[in] aSourceSurface` | A handle to the surface from which you want to bitblt. |
| | `[in] aSourceRect` | The area in the source surface from which you want to bitblt. |
| | `[in] aAlphaMask` | A bitmap containing per-pixel alpha values |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to a surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – aDestPoint or aSourceRect is NULL<br><br>twGfxErrorInvalidCoord – the coordinate values in the destination point or the source rectangle address values outside their respective surfaces<br><br>twGfxErrorOperationInProgress – an asynchronous operation is in progress | |
| **Side Effects** | | |
| **Comments** | The alpha mask bitmap provides a per-pixel alpha value to perform the blending calculation with (see `TwGfxBlendBlt` for the blending calculation). The alpha mask pixel format must be one of the index formats (monochrome, 2bpp, 4bpp or 8bpp) with the 4bpp being the most efficient choice.<br><br>Each pixel of the source surface in the source rectangle is blended with the destination pixel using the blending formula described in TwGfxBlendBlt. The alpha color value is taken from the mask, which means that each pixel can have a different alpha value applied to the calculation. | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxStretchBlt**

| | | |
|---|---|---|
| **Purpose** | Basic bitblt rendering call. | |
| **Prototype** | `Err TwGfxStretchBlt(TwGfxSurfaceType* aDestSurface, const TwGfxRectType* aDestRect, TwGfxSurfaceType* aSourceSurface, const TwGfxRectType* aSourceRect)` | |
| **Fields** | `[in] aDestSurface` | A handle to the surface to which you want bitblt. |
| | `[in] aDestRect` | The area in the destination surface to which you want bitblt. |
| | `[in] aSourceSurface` | A handle to the surface from which you want to bitblt. |
| | `[in] aSourceRect` | The area in the source surface from which you want to bitblt. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to a surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – aDestPoint or aSourceRect is NULL<br><br>twGfxErrorInvalidCoord – the coordinate values in the destination point or the source rectangle address values outside their respective surfaces<br><br>twGfxErrorOperationInProgress – an asynchronous operation is in progress | |
| **Side Effects** | | |
| **Comments** | The source surface is *stretched* or *shrunk* to fit in the destination rectangle. The exact effect on the pixels is not defined here. This operation is not fast, but is marginally accelerated by the graphics accelerator. | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxTileBlt**

| | | |
|---|---|---|
| **Purpose** | Basic bitblt rendering call. | |
| **Prototype** | `Err TwGfxTileBlt(TwGfxSurfaceType* aDestSurface, const TwGfxRectType* aDestRect, TwGfxSurfaceType* aSourceSurface, const TwGfxPointType* aSourceAlignmentPoint)` | |
| **Fields** | `[in] aDestSurface` | A handle to the surface to which you want bitblt. |
| | `[in] aDestRect` | The destination area to tile with the source surface. |
| | `[in] aSourceSurface` | A handle to the surface from which you want to bitblt. |
| | `[in] aSourceAlignmentPoint` | The offset in x & y from which to begin the tiling. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to a surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – aDestPoint or aSourceRect is NULL<br><br>twGfxErrorInvalidCoord – the coordinate values in the destination point or the source rectangle address values outside their respective surfaces<br><br>twGfxErrorOperationInProgress – an asynchronous operation is in progress | |
| **Side Effects** | | |
| **Comments** | The source surface is drawn as many times as necessary to cover the destination rectangle. The first pixel written to the upper-left corner of the destination rectangle will come into the source surface from the aSourceAlignmentPoint (x,y) offset. | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxTransformBlt**

| | | |
|---|---|---|
| **Purpose** | Transforming bitblt rendering call. | |
| **Prototype** | `Err TwGfxTransformBlt(TwGfxSurfaceType* aDestSurface, const TwGfxPointType* aDestPoint, TwGfxSurfaceType* aSourceSurface, const TwGfxRectType* aSourceRect, Int32 aRotationFlags, Int32 aMirrorFlags)` | |
| **Fields** | `[in] aDestSurface` | A handle to the surface to which you want bitblt. |
| | `[in] aDestPoint` | The upper-left coordinate in the destination surface to which you want bitblt. |
| | `[in] aSourceSurface` | A handle to the surface from which you want to bitblt. |
| | `[in] aSourceRect` | The area in the source surface from which you want to bitblt. |
| | `[in] aRotationFlags` | A value indicating the kind of rotation to perform. |
| | `[in] aMirrorFlags` | A value indicating the kind of mirroring to perform. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to a surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – aDestPoint or aSourceRect is NULL<br><br>twGfxErrorInvalidCoord – the coordinate values in the destination point or the source rectangle address values are outside their respective surfaces<br><br>twGfxErrorInvalidRotation – the rotation value is invalid<br><br>twGfxErrorInvalidMirror – the mirror value is invalid<br><br>twGfxErrorOperationInProgress – an asynchronous operation is in progress | |
| **Side Effects** | | |
| **Comments** | Legal values for aRotationFlags:<br>twGfxRotateNone, twGfxRotateCW90, twGfxRotateCW180, twGfxRotateCW270, twGfxRotateCCW90, twGfxRotateCCW180, twGfxRotateCCW270<br>Legal values for aMirrorFlags:<br>twGfxMirrorNone, twGfxMirrorHorizontal, twGfxMirrorVertical, twGfxMirrorBoth | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxDrawPoints**

| | | |
|---|---|---|
| **Purpose** | Draw a set of points to the destination surface. | |
| **Prototype** | `Err TwGfxDrawPoints(TwGfxSurfaceType* aDestSurface, const TwGfxPointType* aPoints, Int32 aNumberOfPoints, TwGfxPackedRGBType aColor)` | |
| **Fields** | `[in] aDestSurface` | A handle to the surface. |
| | `[in] aPoints` | A pointer to the array of TwGfxPointType objects containing the x,y coordinates of the points to be plotted. |
| | `[in] aNumberOfPoints` | The number of points to plot. |
| | `[in] aColor` | The color to use when plotting each point. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to the surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – aPoints is NULL<br><br>twGfxErrorInvalidCount – the aNumberOfPoints value is <= zero<br><br>twGfxErrorOperationInProgress – an asynchronous operation is in progress | |
| **Side Effects** | | |
| **Comments** | | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

### TwGfxDrawColorPoints

| | | |
|---|---|---|
| **Purpose** | Draw a set of points to the destination surface. Each point has its own color. | |
| **Prototype** | `Err TwGfxDrawColorPoints(TwGfxSurfaceType* aDestSurface, const TwGfxPointType* aPoints, Int32 aNumberOfPoints, const TwGfxPackedRGBType* aColors)` | |
| **Fields** | `[in] aDestSurface` | A handle to the surface. |
| | `[in] aPoints` | A pointer to the array of TwGfxPointType objects containing the x,y coordinates of the points to be plotted. |
| | `[in] aNumberOfPoints` | The number of points to plot. |
| | `[in] aColors` | The color to use when plotting each point. There must be one color in the aColors array for each point in the aPoints array. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to a surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – aPoints or aColors is NULL<br><br>twGfxErrorInvalidCount – the aNumberOfPoints value is <= zero<br><br>twGfxErrorOperationInProgress – an asynchronous operation is in progress | |
| **Side Effects** | | |
| **Comments** | | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxDrawLines**

| | | |
|---|---|---|
| **Purpose** | Draw one or more connected lines. | |
| **Prototype** | `Err TwGfxDrawLines(TwGfxSurfaceType* aDestSurface, const TwGfxPointType* aPoints, Int32 aNumberOfPoints, TwGfxPackedRGBType aColor)` | |
| **Fields** | `[in] aDestSurface` | A handle to the surface. |
| | `[in] aPoints` | The set of points that define the lines to be drawn. Lines are drawn starting at (aPoints[I-1].x,aPoints[I-1].y) to (aPoints[I].x,aPoints[I].y) where I goes from 1 to aNumberOfPoints-1. Therefore, if aNumberOfPoints is 3 then 2 lines will be drawn. |
| | `[in] aNumberOfPoints` | The number of points in aPoints. |
| | `[in] aColor` | The color to use when drawing each line. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to a surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – aPoints is NULL<br><br>twGfxErrorInvalidCount – the number of points is less than two.<br><br>twGfxErrorOperationInProgress – an asynchronous operation is in progress | |
| **Side Effects** | | |
| **Comments** | Example: If aNumberOfPoints is 3 then the following two lines are drawn:<br><br>`(aPoints[0].x,aPoints[0].y) to (aPoints[1].x,aPoints[1].y)`<br><br>`(aPoints[1].x,aPoints[1].y) to (aPoints[2].x,aPoints[2].y)` | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxDrawLineSegments**

| | | |
|---|---|---|
| **Purpose** | Draw one or more independent lines. | |
| **Prototype** | `Err TwGfxDrawLineSegments(TwGfxSurfaceType* aDestSurface, const TwGfxPointType* aPoints, Int32 aNumberOfPoints, TwGfxPackedRGBType aColor)` | |
| **Fields** | `[in] aDestSurface` | A handle to the surface. |
| | `[in] aPoints` | The set of points that define the lines to be drawn. |
| | `[in] aNumberOfPoints` | The number of points in aPoints. |
| | `[in] aColor` | The color to use when drawing each line. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to a surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – aPoints is NULL<br><br>twGfxErrorInvalidCount – the number of points is not even or is <= zero<br><br>twGfxErrorOperationInProgress – an asynchronous operation is in progress | |
| **Side Effects** | | |
| **Comments** | Example: If aNumberOfPoints is 4 then the following two lines are drawn:<br><br>`(aPoints[0].x,aPoints[0].y) to (aPoints[1].x,aPoints[1].y)`<br><br>`(aPoints[2].x,aPoints[2].y) to (aPoints[3].x,aPoints[3].y)` | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

### TwGfxDrawRect

| | | |
|---|---|---|
| **Purpose** | Draw the outline of a rectangle. | |
| **Prototype** | `Err TwGfxDrawRect(TwGfxSurfaceType* aDestSurface, const TwGfxRectType* aRect, TwGfxPackedRGBColor aColor)` | |
| **Fields** | `[in] aDestSurface` | A handle to the surface. |
| | `[in] aRect` | A pointer to the rectangle to outline. |
| | `[in] aColor` | The color to use when drawing the outline. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to a surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – aRect is NULL<br><br>twGfxErrorOperationInProgress – an asynchronous operation is in progress | |
| **Side Effects** | | |
| **Comments** | | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxFillRect**

| | | |
|---|---|---|
| **Purpose** | Fill a rectangle with the given color. | |
| **Prototype** | `Err TwGfxFillRect(TwGfxSurfaceType* aDestSurface, const TwGfxRectType* aRect, TwGfxPackedRGBColor aColor)` | |
| **Fields** | `[in] aDestSurface` | A handle to the surface. |
| | `[in] aRect` | A pointer to the rectangle to fill. |
| | `[in] aColor` | The color to use when filling the rectangle. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to a surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – aRect is NULL<br><br>twGfxErrorOperationInProgress – an asynchronous operation is in progress | |
| **Side Effects** | | |
| **Comments** | | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxDrawSpans**

| | | |
|---|---|---|
| **Purpose** | Draw one or more horizontal spans with a constant color. | |
| **Prototype** | `Err TwGfxDrawSpans(TwGfxSurfaceType* aDestSurface, const TwGfxSpanType* aSpans, Int32 aNumberOfSpans, TwGfxPackedRGBColor aColor)` | |
| **Fields** | `[in] aDestSurface` | A handle to the surface. |
| | `[in] aSpans` | A pointer to one or more TwGfxSpanType objects. |
| | `[in] aNumberOfSpans` | The number of spans to draw. |
| | `[in] aColor` | The color to draw each span. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to a surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – aSpans is NULL<br><br>twGfxErrorInvalidCount – the number of spans is <= zero<br><br>twGfxErrorOperationInProgress – an asynchronous operation is in progress | |
| **Side Effects** | | |
| **Comments** | | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |

**TwGfxDrawBitmap**

| | | |
|---|---|---|
| **Purpose** | Draw a bitmap to the destination surface. | |
| **Prototype** | `Err TwGfxDrawBitmap(TwGfxSurfaceType* aDestSurface, const TwGfxPointType* aDestPoint, const TwGfxBitmapType* aBitmap)` | |
| **Fields** | `[in] aDestSurface` | A handle to the surface. |
| | `[in] aDestPoint` | The destination coordinates in the destination surface (upper-left) to draw the bitmap. |
| | `[in] aBitmap` | A pointer to the TwGfxBitmapType object which describes the size and pixel format of the bitmap. Note that the size field must be set to the size of the TwGfxBitmapType data structure. |
| **Result** | `errNone` – Succeeded<br><br>twGfxErrorInvalidHandle – the handle to a surface is invalid or the library is not open<br><br>twGfxErrorNullPointer – aDestPoint or aBitmap is NULL<br><br>twGfxErrorBadObjectVersion – the TwGfxBitmapType size field doesn't match a known version for the library<br><br>twGfxErrorOperationInProgress – an asynchronous operation is in progress | |
| **Side Effects** | | |
| **Comments** | The following pixel formats are supported for bitmaps:<br><br>twGfxPixelFormatMonochrome<br>twGfxPixelFormat2bpp<br>twGfxPixelFormat4bpp<br>twGfxPixelFormat8bpp<br>twGfxPixelFormatRGB565_LE<br>twGfxPixelFormatRGB565_BE<br><br>For index formats the palette field in aBitmap must point to a table of color values used to convert the index data into display pixels. | |
| **Header** | `TwGfx.h` | |
| **Sample** | `TBD` | |